

# Driving Platform Adoption with Embedded SREs

Leena Mooneeram (Chainalysis)  
Jorge Lainfiesta (Rootly)

Story time



Platform Engineering  
Challenge

# Adoption (& Alignment)

# SRE is a more established practice

- 20 years since Google coined the term and published the books
- No stakeholder will doubt the role of reliability

# Proactive reliability struggles

- SRE teams are usually small, very efficient
- But often lack the resources to do large-scale proactive initiatives
- Stuck on fire fighting mode too often

Jorge  
Lainfiesta  
Reliability Advocate



Leena  
Mooneeram  
Senior Platform Engineer



10K Medalist



# Setting the Scene - Out of the Box

- We don't have a dedicated SRE team at Chainalysis.
- We have a central Platform team and a number of embedded devops engineers
- We provide O11y, and a base for reliability, out of the box - for new services.



What about the  
Older, Bigger,  
(more scary?)  
services...

- Retrofitting the golden path isn't easy
- Self-Service isn't an option



# Why older services hesitate...

- The perceived risk can be too high
- The perceived investment can seem too expensive
  - *especially when teams are focused on keeping their customers happy*
- The tech-debt mountain may just look too high to climb



# What's a Platform team to do?



Send in the Platform-Team

We thought we'd learn from our SRE colleagues

We sent some Platform Engineers to select teams - SRE  
embedded-style, for a time-limited mission

# Can Platform learn from SREs?

- Overview the relationship between SRE and Platform Engineering
- How we trailed an embedded Platform SRE program
- What we learned on each iteration of the program
- What does the road ahead look like?

# 1: Picked the Easy Services

- We made it easy for ourselves.
- We got great results. Easy to migrate.
- Effective, but didn't prepare us for what really delivers value



## 2: Taught by Example

The Platform team were Driving

We delivered;

- Guided migrations
- Demonstrations of moving closer towards alignment
- Guides for self-service





## 2: Taught by Example

*The Platform team were Driving*

We got;

- Low attendance
- Limited Engagement
- Frustration (for everyone)



### 3: Taught a Team to Fish

Give someone a fish and you feed them for a day;

Teach someone to fish and you feed them for a lifetime.

... at least that was the plan.

We:

1. Handed over the keys to the Feature teams
2. Let the Feature teams drive





### 3: Taught a Team to Fish

We got;

1. Higher engagement
2. More involvement
3. Less frustration



## 4. Single Point of Ownership

- Exceptional performance. Biggest impact.

Even though engagement was great and people were learning, the migration progress was going very slow and driven exclusively by us and our mob sessions.

- Accountability was key to drive service migrations.

## 5. Senior leaders (active) Sponsorship

# So Where are We Now?

- We're in the middle.
- Teams are migrating their services to the managed platform - with a varying levels of support from Platform

# But this is just the beginning

We're still learning

- We thought that all teams just needed to learn `how to fish`
- Lessons didn't always translate between teams

 Maybe this is a human challenge not a technical one?

# What we learned?

There isn't a Silver Bullet - one size doesn't fit all

We need to approach it as engineers.

- Agile & Iterative. If something doesn't work, move on to the next.

At best, aim to have

- A set of approaches to use.



We need everybody to want the same thing

# What did work?

- Having a strong Platform presence
- Time limited well-scoped engagement with application teams
- Single Owner
- Active & Visible Senior Leadership Sponsorship

New Service

Yes

Golden path/Self-service

- Defaults baked in
- Path of least resistance
- Light support

No

Step 2: Strong DevOps/SRE expertise?

Yes

Empower and accelerate

- Guided onboarding or short embed
- Treat as accelerator team
- Provide extra support

No

Step 3: Tried before but stalled

Yes

Recovery coaching

- Diagnose blockers
- Unblock issues
- Restart momentum

No

Step 4: Low expertise but willing & time now?

Yes

Support while they drive

- They do the work
- Platform/SRE pair and review
- Build ownership and capability

No

Step 5: Unwilling or no time

Yes

Step 6: Is the service critical?

No

Proceed with planned adoption path

After any branch

Yes

Document success as an internal case study

No

Step 6: Did adoption stick?

- Escalate blockers
- Leadership buy-in
- Incentives
- Workload

Leena Mooneeram



Jorge Lainfiesta



Critical and stable

- Embedded SRE mission (2-6 wks)
- Bootstrap reliability
- Clear exit/handover

Critical but stable

- Reassess drivers
- If blocking platform work, do a platform-led migration sweep
- If not blocking, deprioritise

Not critical

- Stabilise first, then adopt
- Add capacity
- Reduce incidents
- Defer feature delivery until stable